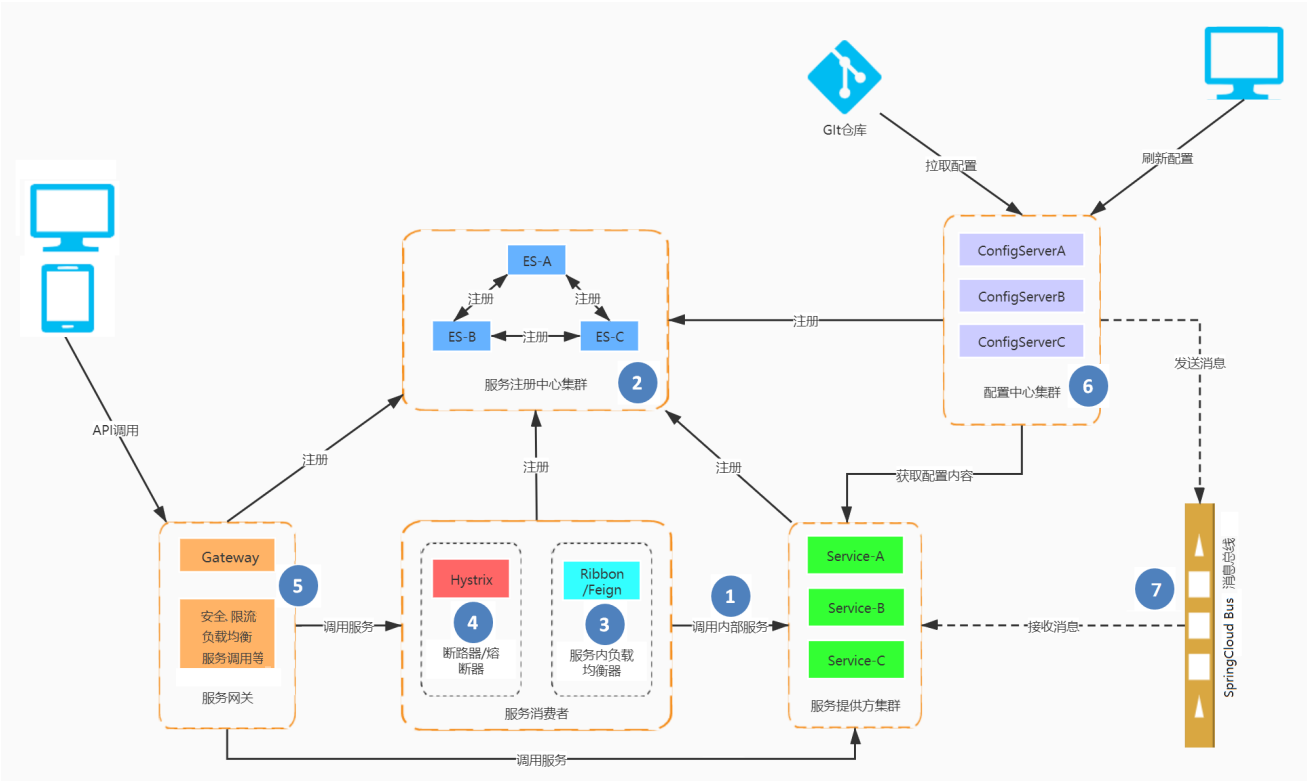
本章所讲内容为 Spring Cloud 架构图中的第三个部分。



1. Ribbon 微服务调用

经过前面的学习，我们已经可以通过服务注册与发现，来进行服务调用。但是服务之间的调用还存在很多的问题，比如：如何更方便的进行服务调用？ 如何对微服务集群实现负载均衡的调用等？

1.1.Ribbon概述

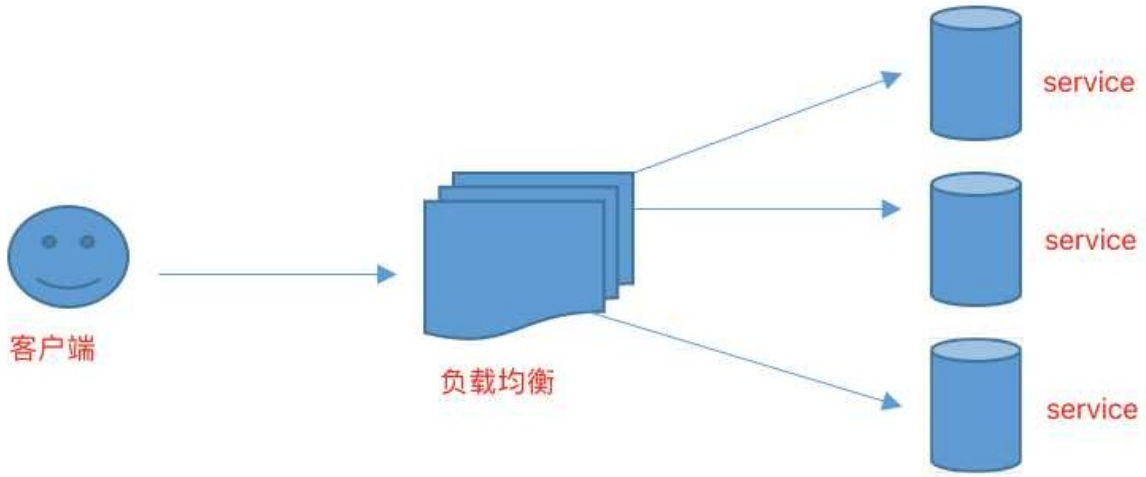
Ribbon，是 Netflix 发布的一个负载均衡器，有助于控制 HTTP 和 TCP客户端行为。在 SpringCloud 中，Eureka 一般配合Ribbon进行使用，Ribbon提供了客户端负载均衡的功能，Ribbon利用从Eureka中读取到的服务信息，在调用服务节点提供的服务时，会合理的进行负载均衡处理。

Ribbon的主要功能：

1. 服务调用：
   基于Ribbon实现服务调用， 是通过拉取到的所有服务列表组成（服务名-请求路径的）映射关系。借助 RestTemplate 最终进行调用。
2. 负载均衡
   当有多个服务提供者时，Ribbon可以根据负载均衡的算法自动的选择需要调用的服务地址。

1.2.Ribbon的负载均衡

什么是负载均衡 负载均衡是一种基础的网络服务，其原理是通过运行在前面的负载均衡服务，按照指定的负载均衡算 法，将流量分配到后端服务集群上，从而为系统提供并行扩展的能力。



负载均衡有两种：客户端负载均衡与服务端负载均衡

1. 服务端负载均衡

   比如nginx。先发送请求到负载均衡服务器或者软件，然后通过负载均衡算法，在多个服务器之间选择一个进行访问；即在服务器端再进行负载均衡算法分配。

2. 客户端负载均衡

   客户端会有一个服务器地址列表，在发送请求前通过负载均衡算法选择一个服务器，然后进行访问，这是客户端负载均衡；即在客户端就进行负载均衡算法分配

Ribbon就是一个典型的客户端负载均衡器。Ribbon会获取所有服务地址，然后根据负载均衡算法，获取本次请求的服务地址。下面使用Ribbon做一个负载均衡的实例。

1.3.Ribbon负载均衡实例

1.3.1.搭建服务提供者集群

1. 在父工程下，创建 Maven Module 子工程（工程名：provider_server_11001；Packaging：jar）
2. 将provider_server_11000工程中的所有代码与配置，都拷贝副本到provider_server_11001工程中。
3. 将provider_server_11001工程配置文件中的端口号改成：11001

```yaml
server:
    port: 11001

spring:
    application:
        name: provider-server      #两个服务名一致（服务名不能使用下划线）

#eureka配置
eureka:
    client:
        service-url:
            #将自己注册给Eureka Server集群
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
```

这样，我们就搭建了一个服务提供者集群。

启动provider_server_11001服务之后，就会在Eureka管理中心中看到集群配置：

Application	AMIs	Availability Zones	Status
CONSUMER-SERVER	n/a (1)	(1)	UP (1) - 10.10.32.54:8888
PROVIDER-SERVER	n/a (2)	(2)	UP (2) - 10.10.32.54:8001 , 10.10.32.54:8002

> 注意：
>
> 1. 此集群中只有两个微服务。
> 2. 两个微服务的功能一致，服务名也一致，但端口是不一致的。

1.3.2.实现Ribbon调用

Eureka已经集成了Ribbon，所以不需要导入任何额外的依赖。

修改服务消费者工程consumer_server_12000 中的主启动类，添加@LoadBalanced注解：

```java
@SpringBootApplication
public class MyApplication {

    /*
     * @LoadBalanced ，是ribbon提供的负载均衡注解。
     * 让RestTemplate在请求时拥有客户端负载均衡的能力
     */
    @LoadBalanced
    @Bean
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);

    }
```

```
}
```

修改服务消费者工程consumer_server_12000 中的控制器：

```
@GetMapping("/getUserById/{userId}")
public CommonResult getUserById(@PathVariable("userId") Integer userId){

    //使用微服务名替换IP地址和端口
    CommonResult result = restTemplate.getForObject(
            "http://provider-server/user/getUserById/"+userId, CommonResult.class);
    return result;
}
```

> 注意：
>
> - Ribbon不支持下划线，只支持中划线。比如：微服务名不能取名为 provider_server。
> - IP地址和端口都不用书写了，因为 provider-server 代表了服务提供者集群中的所有节点。Ribbon会在这些节点中进行负载均衡的调用。

1.3.3.测试负载均衡

测试时，为了能够清晰的看到调用的是哪一个服务，我们将服务器提供者响应数据修改一下：

```
//provider_server_11000工程中的控制器修改如下：
return new CommonResult(200,"success(11000)",new User(userId,"张三","123"));

//provider_server_11001工程中的控制器修改如下：
return new CommonResult(200,"success(11001)",new User(userId,"张三","123"));
```

启动服务后可以看到，两个服务提供者轮询提供服务。

{"code":200,"message":"success(11000)","result":{"userId":22,"userName":"张三","password":"123"}}

1.4.负载均衡策略

Ribbon内置了多种负载均衡策略：

1. 轮询策略（com.netflix.loadbalancer.RoundRobinRule）
2. 随机策略（com.netflix.loadbalancer.RandomRule）
3. 重试策略（com.netflix.loadbalancer.RetryRule）：在一个配置时间段内（超时时间），当选择服务实例不成功，则一直尝试选择一个可用的服务实例。
4. 权重策略（com.netflix.loadbalancer.WeightedResponseTimeRule）：会计算每个服务的权重，越高的被调用的可能性越大。
5. 最佳策略（com.netflix.loadbalancer.BestAvailableRule）：遍历所有的服务实例，过滤掉故障实例，并返回请求数最小的实例。

6. 可用过滤策略（com.netflix.loadbalancer.AvailabilityFilteringRule）：过滤掉故障和请求数超过阈值的服务实例，再从剩下的实力中轮询调用。

负责均衡策略是可以通过配置文件进行修改的。在服务消费者consumer_server_12000 的application.yml配置文件中修改负载均衡策略：

```
#需要调用的微服务名称
provider-server:
    ribbon:
        #设置使用哪一种负载均衡策略类（这里使用随机策略类）
        NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RandomRule
```

重新启动consumer_server_12000后，可以看到，这次是随机选择不同的服务提供者。

2.基于Feign的服务调用

2.1.Feign简介

前面我们使用的RestTemplate实现REST API调用，代码大致如下：

```
restTemplate.getForObject("http://provider-server/user/getUserById/"+userId,CommonResult.class);
```

由代码可知，我们是使用拼接字符串的方式构造URL的。此种方式，在有多个请求参数时就非常麻烦。此时可以使用Feign来帮助我们更快捷、更优雅的调用HTTP API。

- Feign是Netflix开发的声明式，模板化的HTTP客户端。它可以使我们快捷、优雅的调用HTTP API。
- 在Spring Cloud中使用Feign进行远程调用时，能获得与调用本地方法一样的编码体验，开发者完全感知不到这是远程方法，更感知不到这是个HTTP请求。
- SpringCloud对Feign进行了增强，也就是OpenFeign。
- OpenFeign支持SpringMVC注解，并整合了Ribbon和Eureka，从而让Feign也具有负载均衡的功能。

2.2.Feign服务调用实例

在服务消费者consumer_server_12000工程中使用Feign进行服务调用

2.2.1.导入OpenFeign依赖

在consumer_server_12000工程中添加OpenFeign的依赖

```
<!--SpringCloud整合的openFeign -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

2.2.2.主启动类添加Feign注解

在主启动类上添加开启Feign支持的注解

```
@SpringBootApplication
//开启Spring Cloud Feign的支持功能
@EnableFeignClients
public class MyApplication {

    //RestTemplate不需要了
    /*
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }
    */

    //... ...
}
```

2.2.3.配置调用接口

创建在Feign中调用微服务的接口

```
package com.neusoft.feign;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.neusoft.po.CommonResult;
import com.neusoft.po.User;

//指定调用的服务名，在yml文件中:spring.application.name: provider-server
@FeignClient(name="provider-server")
public interface UserFeignClient {

    //配置需要调用的挂号服务接口。与UserController中的方法定义一致
    @GetMapping("/user/getUserById/{userId}")
    public CommonResult getUserById(@PathVariable("userId") Integer userId);

}
```

> 我们知道，最终调用微服务的url是：
>
> ```
> "http://provider-server/user/getUserById/"+userId
> ```
>
> 而在OpenFeign接口中，会使用@FeignClient注解中的name值，与@GetMapping注解中的value值，共同拼接出与上面一样的url。

2.2.4.使用Feign接口进行调用

在控制器CartController中注入Feign接口，并使用它进行服务调用

```
@RestController
@RequestMapping("/cart")
public class CartController {

    //注入Fegin接口（@EnableFeignClients自动扫描@FeignClient注解）
    @Autowired
    private UserFeignClient userFeignClient;

    @GetMapping("/getUserById/{userId}")
    public CommonResult getUserById(@PathVariable("userId") Integer userId){
        //使用Fegin接口进行服务调用
        return userFeignClient.getUserById(userId);
    }
}
```

2.2.5.测试

启动服务，测试服务调用，同时也可以测试负载均衡（Feign也封装了Ribbon的负载均衡功能）。